Université TÉLUQ

INF1430 — Projet de fin d'études

Travail noté 2

Rapport intermédiaire mi-parcours — Veridian

Étudiant : Francis Côté

Cours : INF1430 — Projet de fin d'études

Institution : Université TÉLUQ

Dépôt : https://github.com/frankydoody/Veridian

Remis conformément aux exigences du cours INF1430

Table des matières

1 Introduction **3**

2 État du tableau de bord GitHub Projects **4**
2.1 Vue Table au moment de la remise 4
2.2 Politique de gestion des branches et des *pull requests* 4

3 Prototype fonctionnel **5**
3.1 Authentification 5
3.2 Tableau de bord 6
3.3 Réunions et projets 7
3.4 Tâches 8
3.5 Mémoire IA 8

4 Description de la solution **9**
4.1 Problématique et proposition de valeur 9
4.2 Vue fonctionnelle d'ensemble 9

5 Modélisation UML **10**
5.1 Diagrammes de cas d'utilisation 10
5.2 Diagramme de classes 10
5.3 Diagramme d'activité — traitement d'une réunion 13
5.4 Diagrammes de séquence 13
5.4.1 Authentification (inscription) 13
5.4.2 Transcription, extraction des décisions et indexation 13
5.4.3 Interrogation de la mémoire (RAG) 13
5.5 Diagramme d'état — cycle de vie d'une réunion 13
5.6 Diagramme de composants 17
5.7 Diagramme de déploiement 17

6 Architecture logicielle et choix technologiques **17**
6.1 Architecture en couches 17
6.2 Justification des choix de frameworks 17
6.3 Conventions de code 20
6.4 Schéma de base de données 20

7 Tests unitaires **21**
7.1 Stratégie de test 21
7.2 Exemples de code 22

8 Blocs complétés — synthèse de la trace d'ingénierie 24
8.1 Bloc 01 — Infrastructure et configuration 24
8.2 Bloc 02 — Authentification JWT 24
8.3 Bloc 03 — Gestion des projets (CRUD) 24
8.4 Bloc 04 — Gestion des réunions (CRUD) 24
8.5 Bloc 05 — Transcription audio (Whisper) 24
8.6 Bloc 06 — Extraction des décisions (Gemini) 25
8.7 Bloc 07 — Mémoire vectorielle (RAG) 25
8.8 Bloc 10 — Frontend : authentification et navigation 25
8.9 Bloc 11 — Frontend : tableau de bord et pages de données 25
8.10 Bloc 12 — Connexion du frontend à l'API réelle (en cours) 25

9 Problèmes rencontrés 26

10 Avancement et planification de la seconde moitié 28
10.1 Métriques d'avancement 28
10.2 Plan des blocs restants 29
10.3 Risques résiduels 29

11 Conclusion 30

1 Introduction

Le présent rapport constitue le livrable intermédiaire (TN2) du projet Veridian dans le cadre du cours INF1430. Il synthétise l'avancement constaté à mi-parcours à partir de la trace d'ingénierie accumulée dans le dépôt GitHub — commits, *pull requests*, branches et historique de fusion — et présente la modélisation UML, l'architecture logicielle, les choix technologiques et le prototype fonctionnel de l'application.

Veridian est une application web de mémoire organisationnelle alimentée par l'intelligence artificielle. Elle enregistre les réunions d'équipe, en transcrit automatiquement le contenu audio, en extrait les décisions prises à l'aide d'un grand modèle de langage, indexe ces décisions et transcriptions dans une mémoire vectorielle interrogeable en langage naturel, et doit éventuellement détecter les contradictions entre décisions successives d'un même projet.

Au moment de la rédaction de ce document, le backend applicatif est fonctionnel de bout en bout : un utilisateur peut s'inscrire, créer un projet, planifier une réunion, téléverser un enregistrement audio, obtenir sa transcription, déclencher l'extraction des décisions qui en découlent, et interroger la mémoire du projet par une question en langage naturel. Le frontend React affiche désormais des données réelles provenant de l'API plutôt que des données statiques, ce travail de connexion étant la dernière étape active avant la fusion du bloc courant.

Avancement global constaté

9 blocs fusionnés dans `main` sur 18 planifiés, 1 bloc en fin de développement actif 50 %

Blocs 01 à 07, 10 et 11 — fusionnés et vérifiés fonctionnels. Bloc 12 — développement terminé sur sa branche dédiée, fusion imminente. Bloc 09 — fonctionnalité de chat opérationnelle, export PDF non débuté. Bloc 08 — fonction de détection de contradiction écrite mais non encore reliée à l'API.

Ce rapport couvre, dans l'ordre : l'état du tableau de suivi GitHub Projects, un aperçu du prototype fonctionnel illustré de captures d'écran réelles, une description de la solution, la modélisation UML complète du système (cas d'utilisation, classes, séquence, état, activité, composants, déploiement), l'architecture logicielle et la justification des choix de frameworks, la stratégie et des exemples de tests unitaires, une synthèse technique bloc par bloc, le journal des problèmes rencontrés, les métriques d'avancement et la planification de la seconde moitié du projet.

2 État du tableau de bord GitHub Projects

2.1 Vue Table au moment de la remise

Conformément à la configuration établie dans le rapport TN1, le tableau de suivi utilise une vue *Table* à cinq champs (*Title*, *Assignees*, *Status*, *Linked pull requests*, *Sub-issues progress*) et ne distingue que deux statuts : *Todo* et *Done*. Il n'existe pas de statut intermédiaire « en cours » sur le tableau lui-même — un bloc dont le développement est avancé mais dont la *pull request* n'est pas encore fusionnée demeure donc affiché *Todo*. Le tableau 1 reproduit fidèlement cet état, y compris cette nuance pour le bloc 12.

Table 1 – État du tableau GitHub Projects (vue Table) au moment de la remise

Bloc	Titre	Status	PR liée
01	Infrastructure et Configuration	Done	— (commit direct)
02	Authentification JWT	Done	— (commit direct)
03	Gestion des Projets CRUD	Done	— (commit direct)
04	Gestion des Réunions CRUD	Done	— (commit direct)
05	Transcription Audio (Whisper)	Done	#12
06	Extraction des décisions (Gemini)	Done	fusionné avec le bloc 07
07	Mémoire vectorielle (RAG)	Done	— (commit direct)
08	Détection contradictions & Alertes	Todo	—
09	Chat mémoire & Export PDF	Todo*	—
10	Frontend Auth	Done	#19
11	Frontend Dashboard	Done	#24
12	Frontend API Connect	Todo**	en préparation
13	Frontend Formulaires	Todo	—
14	Frontend Réunion Détaillée	Todo	—
15	Frontend Décisions & Contradictions	Todo	—
16	Frontend Calendrier de projet	Todo	—
17	Tests d'intégration & Qualité	Todo	—
18	Déploiement & Documentation finale	Todo	—

* Le sous-volet « chat mémoire » est fonctionnel (endpoint `POST /api/memory/chat`) ; l'export PDF n'est pas débuté. ** Développement terminé sur la branche `bloc-12` ; statut *Todo* jusqu'à la fusion de la *pull request*.

2.2 Politique de gestion des branches et des *pull requests*

Chaque bloc suit le cycle de vie suivant : création d'une branche depuis `main`, développement, ouverture d'une *pull request*, et fusion par *merge commit* exclusive-

ment (`gh pr merge -merge`). Conformément à la politique de traçabilité du projet (Atlassian, 2024), aucune branche source n'est supprimée après fusion — chaque étape de développement reste consultable indépendamment dans l'historique Git. Cette discipline explique la présence, dans le dépôt, de branches distantes antérieures telles que `branch/bloc-05-transcription` ou `branch/bloc-10-frontend-auth`, conservées malgré leur fusion.

Deux particularités de l'historique réel méritent d'être consignées par souci d'exactitude :

— Le contenu initialement développé sur la branche `branch/bloc-06-decisions` a été recréé directement sur la branche du bloc 07 (commit `cf1e255`) puis fusionné dans `main` avec celle-ci (commit de fusion `83bac52`). Les deux fonctionnalités — extraction des décisions et mémoire vectorielle — sont donc arrivées dans `main` au sein du même *merge commit*, bien qu'elles correspondent à deux blocs distincts du plan de développement.

— Le bloc 10 (frontend auth) a été intégré à `main` une première fois par fusion directe (commit `7172a11`), puis consolidé par la *pull request* #19 (commit de fusion `d64a023`). Les deux écritures reflètent une itération du flux de travail Git en début de phase frontend plutôt qu'une anomalie de contenu.

3 Prototype fonctionnel

Cette section présente l'état visuel réel de l'application au moment de la rédaction, sous la forme de captures d'écran prises directement sur l'interface en exécution locale (`localhost:5173`), après la connexion du frontend aux endpoints réels de l'API (bloc 12). Les écrans sont volontairement présentés sur un compte fraîchement créé, sans données de démonstration : les états vides (« Aucune réunion trouvée », compteurs à zéro) confirment que les pages interrogent bel et bien l'API plutôt que d'afficher des données statiques codées en dur, ce qui constitue la preuve visuelle la plus directe de l'achèvement du bloc 12.

3.1 Authentification

La page de connexion (figure 1) présente le formulaire d'authentification : champ courriel, champ mot de passe, et lien vers la création de compte. L'authentification repose sur l'endpoint `POST /api/auth/login`, qui retourne un jeton JWT persisté côté client. Le menu utilisateur (figure 2), accessible depuis l'en-tête de toutes les pages protégées, affiche le nom et le courriel de la session active ainsi que l'option de déconnexion.

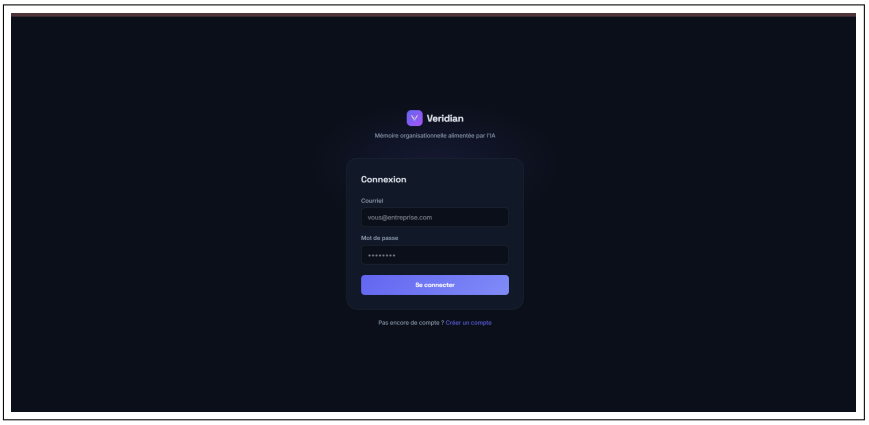


Figure 1 – Page de connexion — formulaire d'authentification

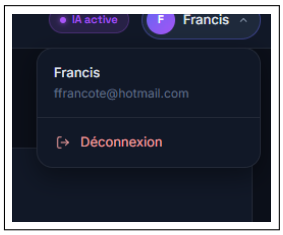


Figure 2 – Menu utilisateur — session active et déconnexion

3.2 Tableau de bord

Le tableau de bord (figure 3.2) constitue la page d'accueil après authentification. Il affiche quatre indicateurs agrégés (réunions du mois, réunions totales, projets actifs, projets totaux) ainsi qu'un flux d'activité récente. Les quatre compteurs à zéro et le message « Aucune activité récente » confirment qu'il s'agit de données réelles issues de la base, et non d'un gabarit statique — un compte neuf affiche naturellement des statistiques nulles.

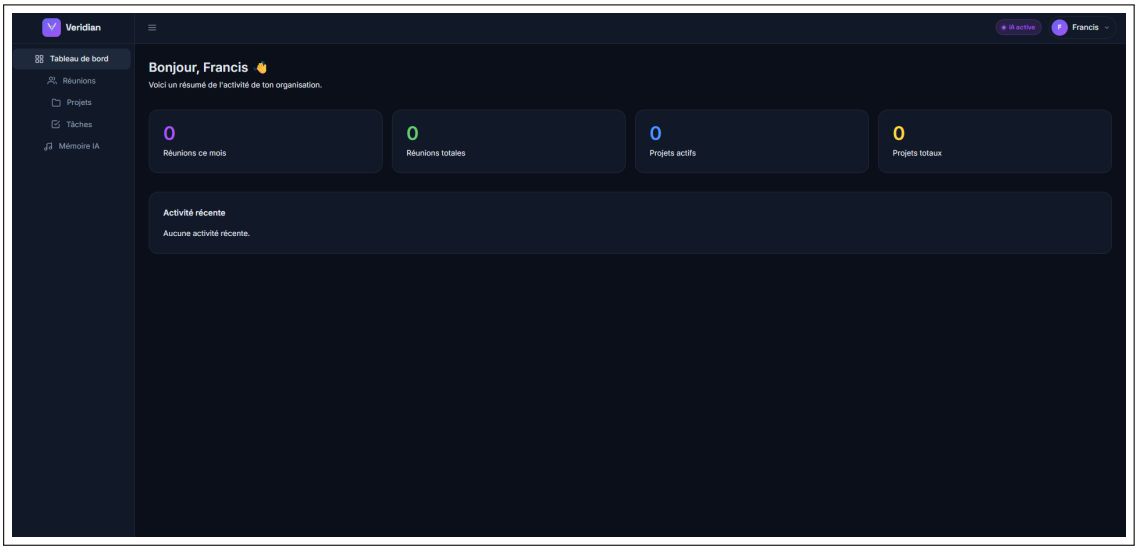


Figure 3 – Tableau de bord — indicateurs agrégés et activité récente

3.3 Réunions et projets

La page Réunions (figure 3.3) et la page Projets (figure 3.3) partagent la même structure : un système de filtres par statut (respectivement *Toutes / En cours / Planifiées / Terminées* et *Tous / Actifs / Archivés / Inactifs*, reflétant les valeurs `CHECK` des colonnes `status` en base), un bouton d'action principal pour la création, et une liste vide confirmant l'absence de données de démonstration.

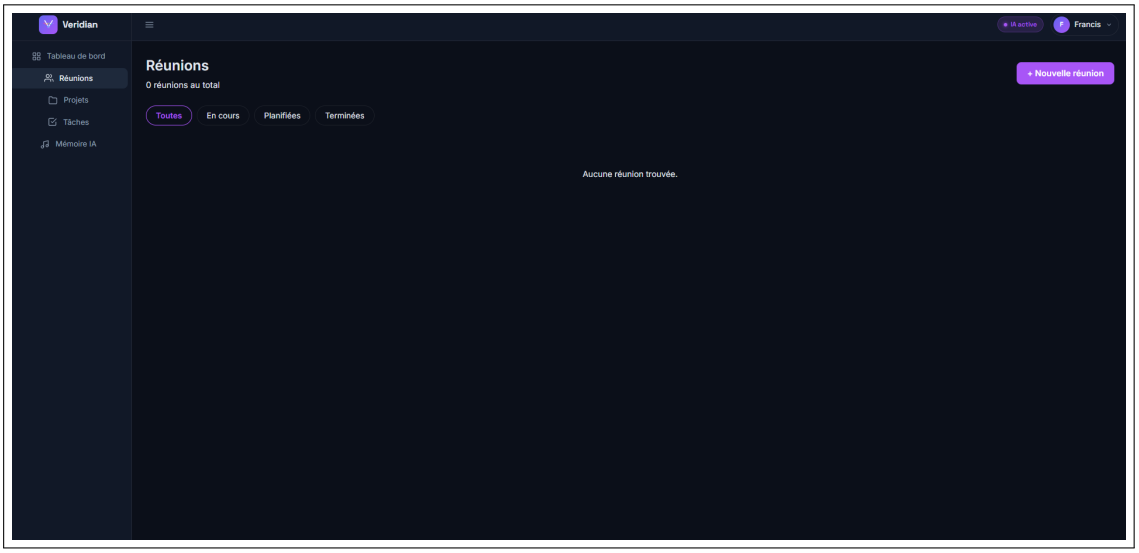


Figure 4 – Page Réunions — filtres par statut et liste vide (API réelle)

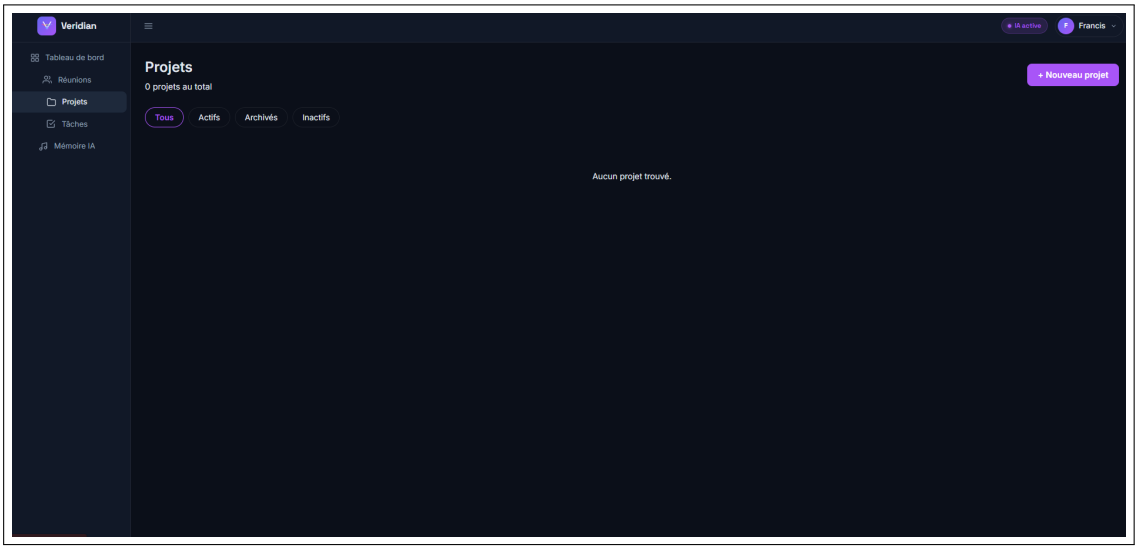


Figure 5 – Page Projets — filtres par statut et liste vide (API réelle)

3.4 Tâches

La page Tâches (figure 3.4) présente un tableau à quatre colonnes (Tâche, Assigné, Projet, Priorité) et précise explicitement, dans son sous-titre, que les tâches sont « extraites automatiquement des réunions » — anticipant l'intégration future de l'extraction automatique par l'IA, alors que la création manuelle (`POST /api/tasks`) est déjà opérationnelle.

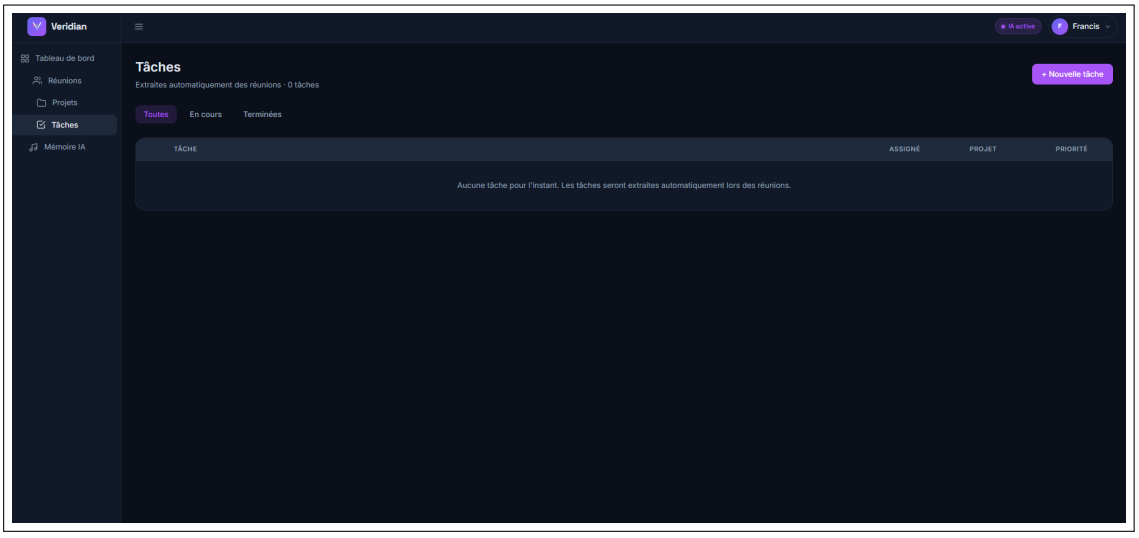


Figure 6 – Page Tâches — tableau et création manuelle

3.5 Mémoire IA

La page Mémoire IA (figure 3.5) est l'interface de chat sur la mémoire vectorielle du projet. Un sélecteur de projet détermine la portée de la recherche sémantique, et un

champ de saisie en bas d'écran permet de poser une question en langage naturel (exemple affiché : *« Quelles décisions ont été prises lors du Sprint 4 ? »*). Cette page consomme l'endpoint `POST /api/memory/chat`, qui orchestre la recherche vectorielle (*retrieval*) puis la génération de réponse par le LLM (*generation*) — le cœur de l'architecture RAG décrite à la section 5.4.3.

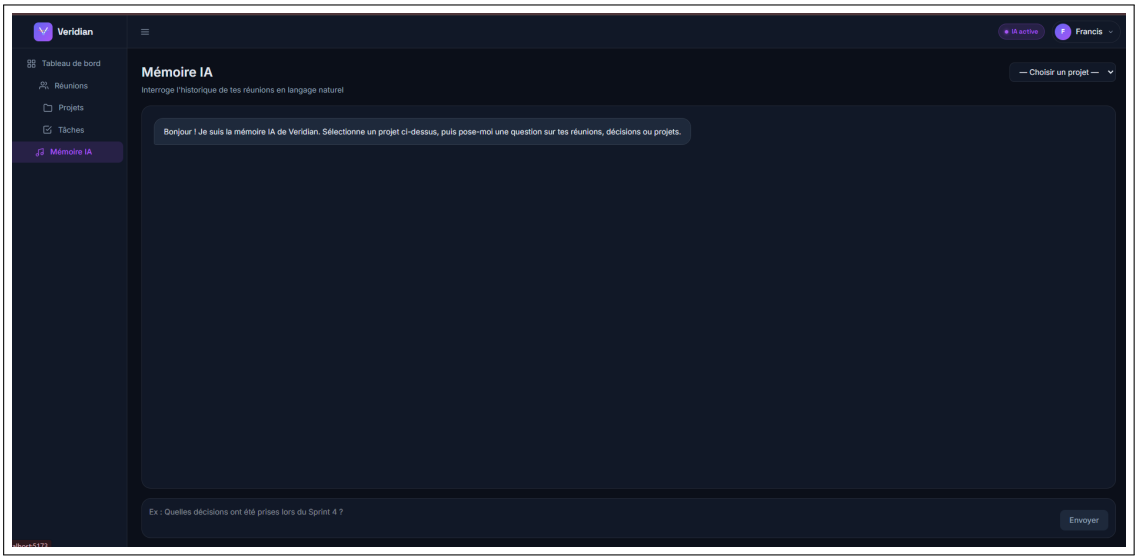


Figure 7 – Mémoire IA — interface de chat RAG sur l'historique du projet

4 Description de la solution

4.1 Problématique et proposition de valeur

Dans une petite organisation, les décisions prises en réunion se dispersent entre notes personnelles, messageries et mémoire individuelle. Il devient difficile, après plusieurs semaines, de retrouver *qui* a décidé *quoi*, *pourquoi*, et si une décision plus récente n'en contredit pas une plus ancienne restée active dans les esprits de certains membres de l'équipe. Veridian répond à ce problème en transformant chaque réunion enregistrée en mémoire organisationnelle structurée, interrogeable et éventuellement auto-vérifiée.

4.2 Vue fonctionnelle d'ensemble

Le parcours utilisateur central de l'application se décompose en quatre étapes fonctionnelles, détaillées dans les diagrammes UML de la section 5 :

1. **Capturer** — un utilisateur crée un projet, planifie une réunion, puis enregistre ou téléverse l'audio de celle-ci.

2. **Comprendre** — le serveur transcrit l'audio (Whisper local) puis en extrait les décisions prises (Gemini), chacune assortie d'un contexte, d'un responsable optionnel, des alternatives écartées et d'un niveau de confiance.
3. **Mémoriser** — la transcription et les décisions sont découpées en segments (*chunks*), vectorisées (*embeddings*), et indexées dans PostgreSQL via l'extension `pgvector`.
4. **Interroger** — un membre de l'équipe pose une question en langage naturel ; le système retrouve les segments les plus pertinents par similarité cosinus et les fournit comme contexte à un LLM pour produire une réponse sourcée.

Une cinquième étape, la **vérification** (détection automatique de contradictions entre une nouvelle décision et les décisions passées d'un même projet), est conçue — la fonction `detectContradiction()` existe dans `ia.service.js` — mais n'est pas encore reliée à un endpoint ni déclenchée automatiquement ; elle correspond au bloc 08, encore au statut *Todo*.

5 Modélisation UML

Cette section présente la modélisation UML du système telle qu'elle reflète le code réellement en place dans le dépôt à mi-parcours — classes, routes, modèles de données et services — plutôt qu'une modélisation prospective. Dix diagrammes sont fournis, couvrant les vues fonctionnelle, structurelle et comportementale du système, conformément aux pratiques de modélisation orientée objet (Fowler, 2018).

5.1 Diagrammes de cas d'utilisation

Les cas d'utilisation sont répartis sur deux diagrammes complémentaires afin de conserver une lecture claire. La figure 8 présente les fonctionnalités directement manipulées par l'*Utilisateur* via le frontend React ; la figure 9 présente le pipeline de traitement d'une réunion, où intervient le *Système IA*, acteur externe composite représentant Whisper (transcription locale) et Gemini (extraction, détection, génération, *embeddings*). Les cas d'utilisation encore au statut *Todo* (export PDF, calendrier, détection de contradictions, alertes) sont représentés en trait pointillé pour les distinguer des fonctionnalités opérationnelles.

5.2 Diagramme de classes

Le diagramme de classes (figure 10) représente le modèle de domaine persistant de Veridian, directement dérivé du schéma relationnel réel (`schema.sql` et migrations). Chaque classe correspond à une table PostgreSQL ; les attributs affichés reprennent



Figure 8 – Diagramme de cas d'utilisation (1/2) — fonctionnalités de gestion et d'interrogation (en pointillé : cas d'utilisation planifiés, non encore implantés)



Figure 9 – Diagramme de cas d'utilisation (2/2) — pipeline de traitement d'une réunion et acteur Système IA (en pointillé : cas d'utilisation planifiés ; l'association de « Recevoir des alertes » avec l'Utilisateur figure sur le diagramme 1/2)

les colonnes effectivement déclarées. Les associations portent les cardinalités réelles imposées par les clés étrangères (`REFERENCES ... ON DELETE RESTRICT`, suppression logique systématique via `is_active`).

5.3 Diagramme d'activité — traitement d'une réunion

Le diagramme d'activité (figure 11) illustre le flux complet déclenché par l'enregistrement d'une réunion, depuis la capture audio jusqu'à la disponibilité des décisions dans la mémoire interrogeable. Il correspond à l'enchaînement réel des appels observés dans `transcription.controller.js` et `decision.controller.js`.

5.4 Diagrammes de séquence

5.4.1 Authentification (inscription)

La figure 12 détaille l'inscription d'un nouvel utilisateur, telle qu'implantée dans `auth.controller.js` : vérification d'unicité du courriel, hachage `bcryptjs`, création de l'enregistrement, puis génération du jeton JWT retourné au client.

5.4.2 Transcription, extraction des décisions et indexation

La figure 13 détaille le pipeline central de l'application, couvrant deux endpoints appelés successivement par le client : le téléversement audio (`POST /api/transcriptions/upload`) puis le déclenchement de l'extraction des décisions (`POST /api/decisions/extract/:meetingId`). Chaque étape d'indexation mémoire est volontairement non bloquante pour le reste du flux (fonctions `*Safely`) : un échec de l'API d'*embeddings* ne fait jamais échouer la transcription ou l'extraction elles-mêmes.

5.4.3 Interrogation de la mémoire (RAG)

La figure 14 détaille le second axe du pipeline RAG : la *récupération* puis la *génération*, déclenchées par `POST /api/memory/chat`. La recherche est systématiquement bornée au périmètre auquel l'utilisateur authentifié a accès (projet ou réunion dont il est membre), avant même d'atteindre la base vectorielle.

5.5 Diagramme d'état — cycle de vie d'une réunion

La figure 15 illustre les transitions d'état de l'entité `Meeting`, telles que contraintes par la colonne `status` (`CHECK (status IN (...))`) du schéma et pilotées par l'endpoint `PATCH /api/meetings/:id/status`. L'état `done` est également atteint automatiquement à la fin d'une transaction d'extraction de décisions (`decision.model.js`, fonction `createDecisions`).

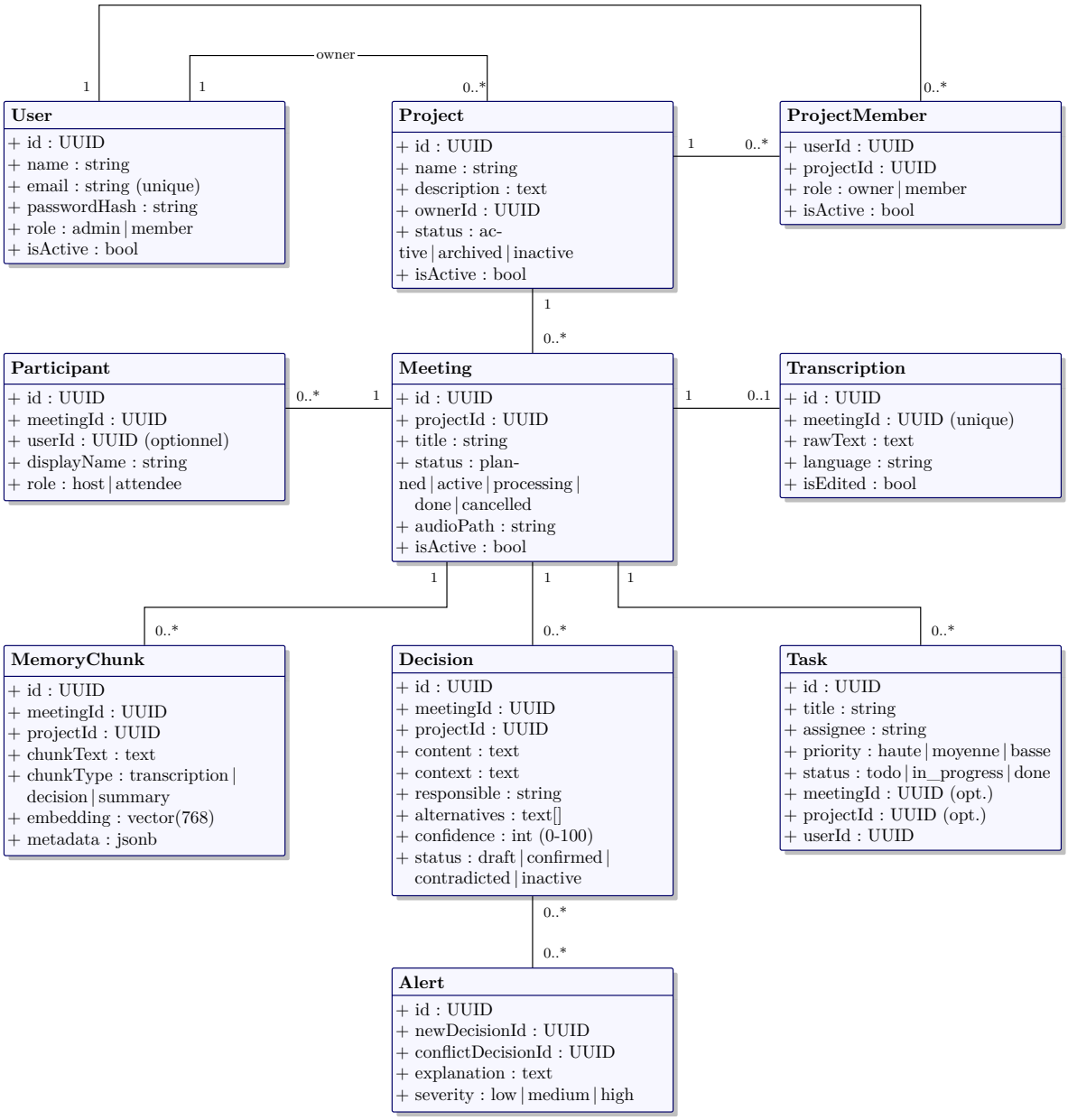


Figure 10 – Diagramme de classes — modèle de domaine persistant (dérivé du schéma PostgreSQL réel)

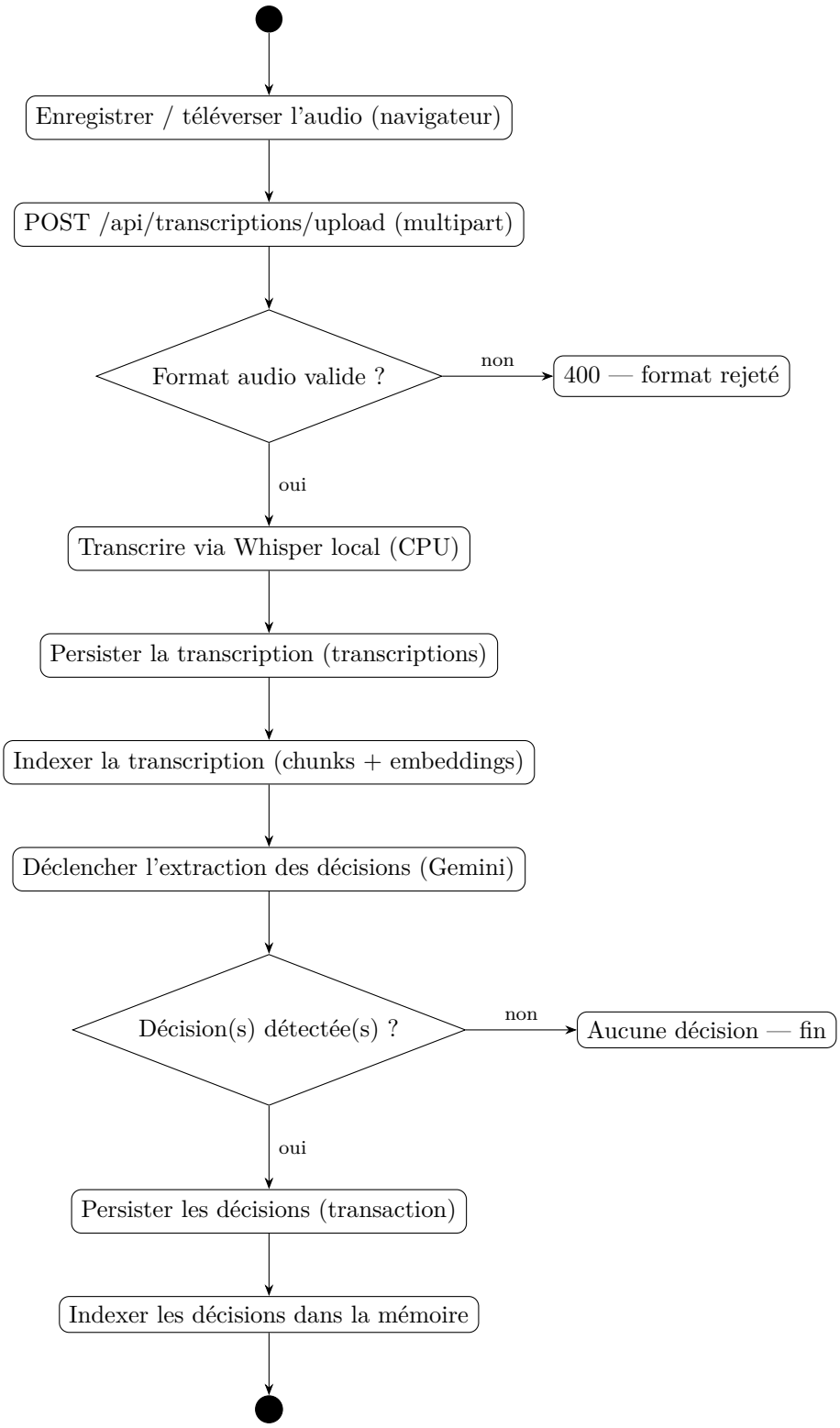


Figure 11 – Diagramme d'activité — pipeline complet de traitement d'une réunion enregistrée

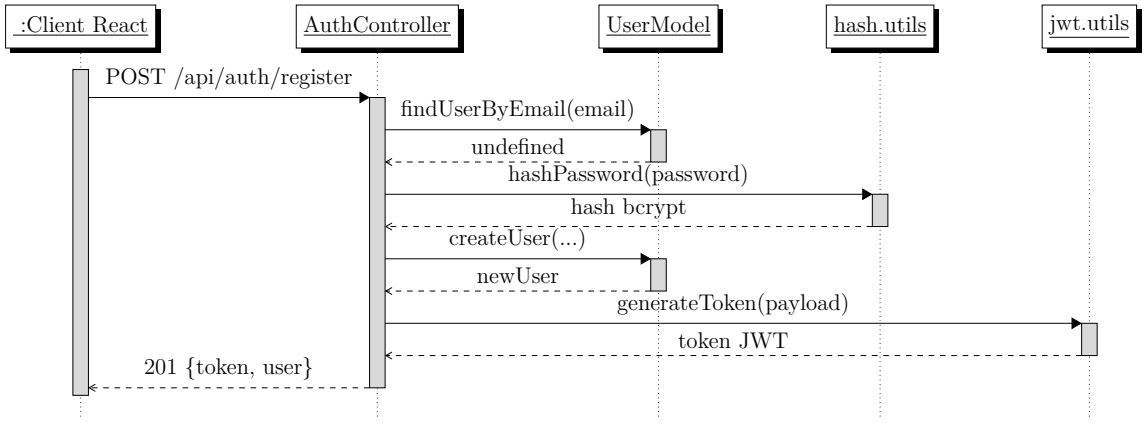


Figure 12 – Diagramme de séquence — inscription d'un utilisateur

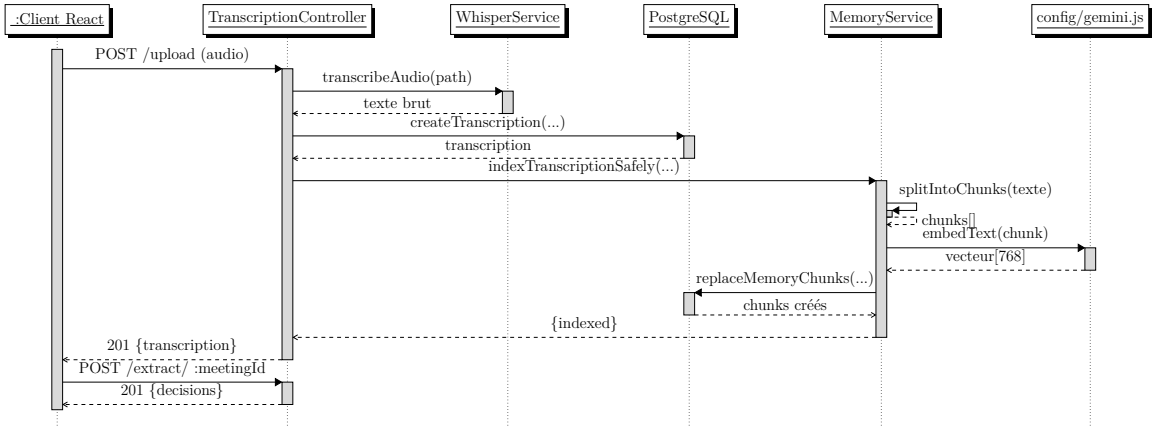


Figure 13 – Diagramme de séquence — transcription audio, indexation et point d'entrée de l'extraction

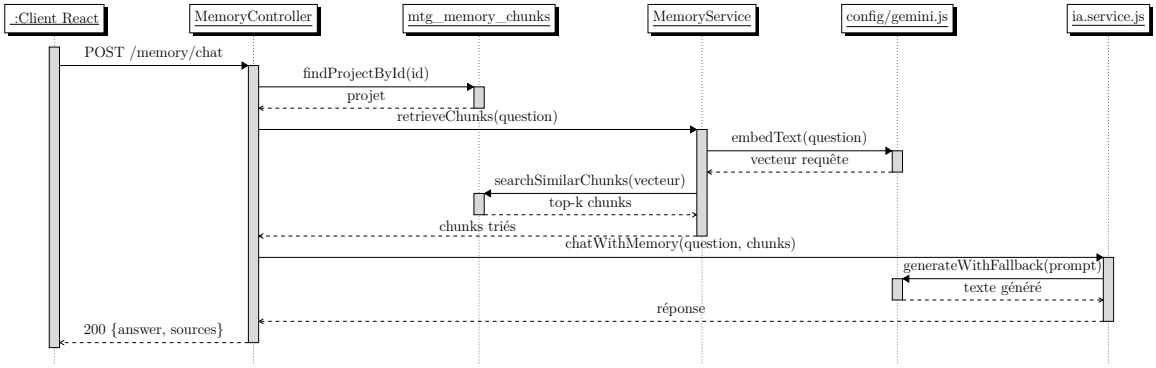


Figure 14 – Diagramme de séquence — interrogation de la mémoire vectorielle (RAG complet)

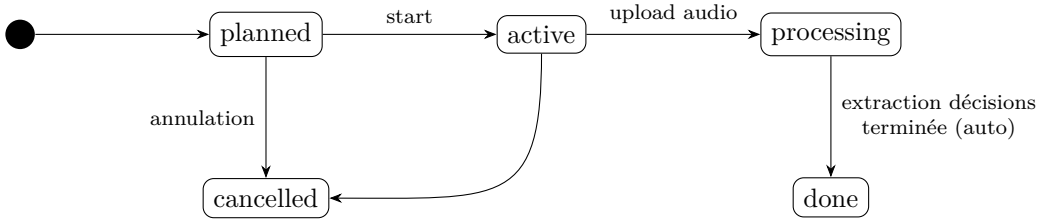


Figure 15 – Diagramme d'état — cycle de vie de l'entité Meeting

5.6 Diagramme de composants

La figure 16 représente la décomposition logicielle en composants déployables et leurs dépendances, reflétant fidèlement l'organisation réelle des répertoires `src/routes`, `src/controllers`, `src/models` et `src/services` côté backend, ainsi que `src/pages`, `src/context` et `src/api` côté frontend.

5.7 Diagramme de déploiement

La figure 17 présente la topologie d'exécution actuelle : un poste de développement unique (Windows ou Ubuntu) hébergeant l'ensemble des processus applicatifs, un conteneur Docker isolé pour la base de données, et deux services externes consommés par HTTPS. Aucun environnement de déploiement distant n'est encore mis en place — cette étape est planifiée au bloc 18.

6 Architecture logicielle et choix technologiques

6.1 Architecture en couches

L'architecture suit un modèle en couches strictement séparées, cohérent de bout en bout entre le frontend React et le backend Express — une discipline qui facilite les tests, limite le couplage, et rapproche le projet des principes d'architecture logicielle en microservices même si Veridian demeure, à ce stade, un monolithe modulaire (Newman, 2021).

Listing 1 – Architecture en couches de Veridian

```
Frontend (React 19 / Vite)
  pages/ -> services/api.js (+ api/client.js) -> AuthContext
        | HTTP / JSON (JWT Bearer)
Backend (Node.js 20 / Express 5)
  routes/ -> middlewares/ -> controllers/ -> models/
                                    \-> services/ (Whisper, IA, memoire)
        | SQL / pgvector
Base de donnees (PostgreSQL 16 + pgvector)
  Docker Compose - conteneur veridian_db
```

6.2 Justification des choix de frameworks

Le tableau 2 justifie chaque choix technologique structurant par rapport au besoin qu'il couvre.

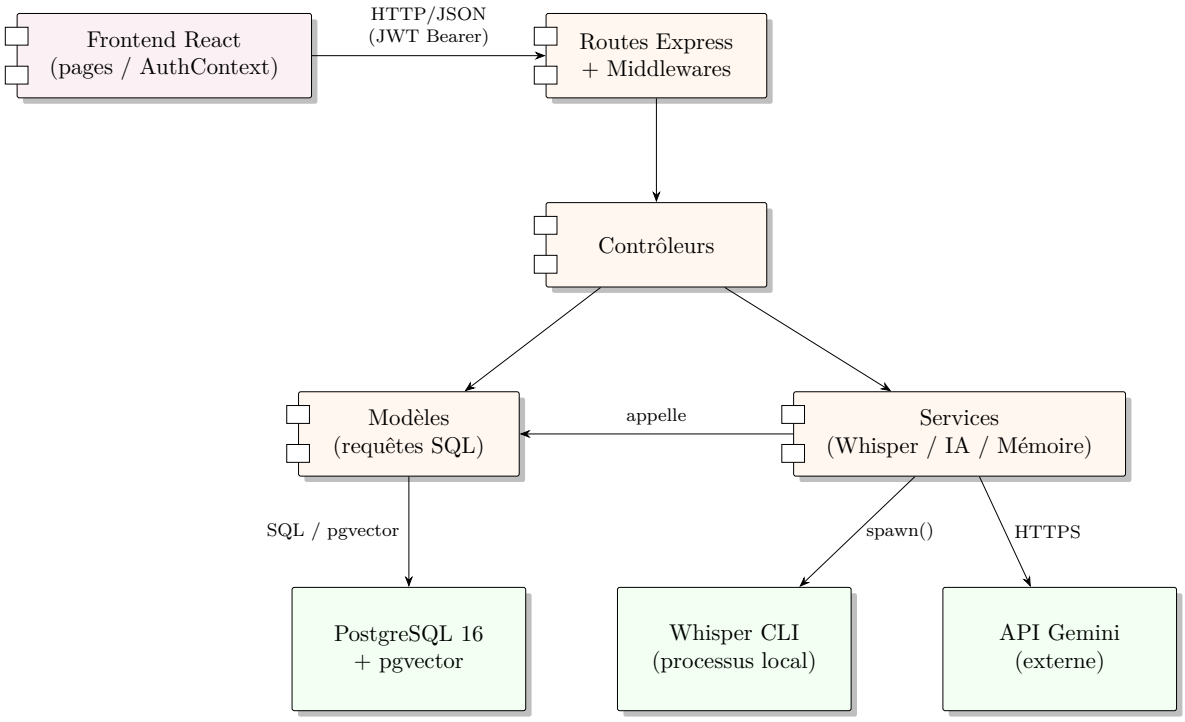


Figure 16 – Diagramme de composants — décomposition logicielle frontend / backend / services externes

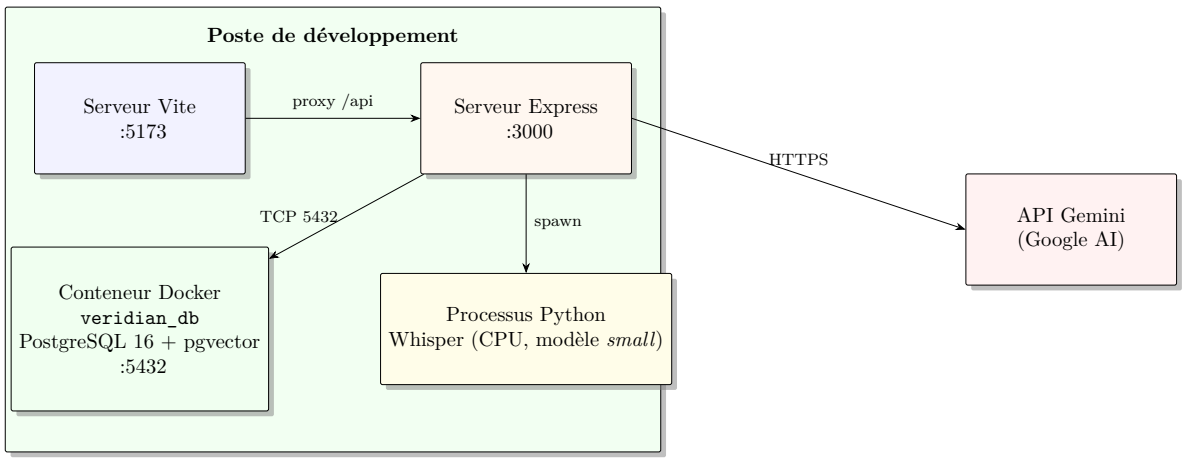


Figure 17 – Diagramme de déploiement — topologie d'exécution en environnement de développement

Table 2 – Choix technologiques et justification

Besoin	**Choix et justification**
API backend	**Node.js 20 / Express 5** — écosystème unique JavaScript du frontend au backend, réduit la charge cognitive pour un développeur seul ; Express 5 apporte la gestion native des promesses rejetées dans les middlewares.
Base de données	**PostgreSQL 16** — SGBD relationnel mature, transactions ACID nécessaires à la cohérence des décisions et des alertes.
Mémoire vectorielle	**pgvector** (PGVECTOR CONTRIBUTORS, 2024) — évite l'introduction d'une base vectorielle dédiée (Pinecone, Weaviate) ; les vecteurs et les données relationnelles cohabitent dans la même transaction et le même moteur de sauvegarde.
Authentification	**JWT (jsonwebtoken) + bcryptjs** (AUTH0 BY OKTA, 2023) — jetons sans état, aucune session serveur à synchroniser, adapté à une architecture qui pourra évoluer vers plusieurs instances.
Transcription	**Whisper local (modèle *small*)** (RADFORD et al., 2022) — exécution locale sans coût récurrent ni dépendance réseau pour cette étape, au prix d'un temps de traitement plus long qu'une API cloud.
Extraction / génération	**Gemini 1.5/2.5 Flash** (GOOGLE DEEPMIND, 2024) — palier gratuit généreux adapté à un projet étudiant, fenêtre de contexte large, *wrapper* `generateWithFallback` ajouté pour basculer automatiquement vers un modèle de secours en cas de surcharge (503) ou de modèle retiré (404).
Frontend	**React 19 / Vite 8 / React Router 7** — recharge à chaud quasi instantanée en développement, écosystème de composants le plus documenté pour un étudiant travaillant seul.
Style	**Tailwind CSS 4** — cohérence visuelle rapide sans fichiers CSS dispersés par composant.
Tests	**Jest 30 + dotenv-cli** — exécuteur de tests standard de l'écosystème Node ; `dotenv-cli` contourne un conflit d'ordre de chargement des variables d'environnement entre ES Modules et `dotenv` lors des tests.
Conteneurisation	**Docker Compose** — reproductibilité de la base de données indépendamment du système d'exploitation (Windows / Ubuntu) utilisé pour développer.

6.3 Conventions de code

Les conventions suivantes, établies dès le bloc 01, sont vérifiées en vigueur dans l'ensemble du code actuel :

— **Suppression logique** — toutes les tables portent un indicateur `is_active` ; aucune instruction `DELETE` SQL n'apparaît dans les modèles, à l'exception volontaire de `task.model.js` (ressource `tasks`, suppression physique assumée car non historisée par design).

— **Transactions dans la couche modèle** — la fonction `withTransaction` (`config/db.js`) encapsule `BEGIN/COMMIT/ROLLBACK` et n'est invoquée que depuis `models/`, jamais depuis un contrôleur. Elle est utilisée pour toute écriture à effets multiples : `createProjectWithOwner`, `createMeetingWithHost`, `createDecisions`, `createMemoryChunks`, `replaceMemoryChunks`.

— **Séparation stricte des couches** — un contrôleur ne contient aucune requête SQL ; un modèle ne contient aucune logique de validation HTTP.

— **IA agnostique** — le fichier `src/services/ia.service.js` ne porte pas le nom du fournisseur ; seul `src/config/gemini.js` connaît l'identité du fournisseur réel, pilotée par les variables `AI_PROVIDER` / `AI_MODEL`.

— **Gestion centralisée des erreurs** — tout contrôleur délègue ses erreurs à `next(error)` ; un middleware unique (`error.middleware.js`) formate la réponse et n'expose la pile d'appel qu'en environnement de développement.

Mise à jour de convention — `dotenv`

Le rapport TN1 documentait un épinglage de `dotenv` à la version 16 pour contourner une incompatibilité avec le chargement des modules ES sur Windows. La version actuellement installée (`package-lock.json`) est `17.4.2` : la contrainte a été levée au cours du développement sans qu'une régression ne soit observée. Cette mise à jour illustre l'utilité du journal des problèmes (section 9) comme trace vivante plutôt que figée.

6.4 Schéma de base de données

Le schéma relationnel compte, au moment de la rédaction, **10 tables** et **11 index explicites** répartis entre `database/schema.sql` et trois fichiers de migration versionnés (`001_update_vector_dimension.sql`, `003_tasks.sql`, `007_memory_chunks.sql`).

Table 3 – Tables PostgreSQL au mi-parcours

Table	Rôle
`users`	Comptes utilisateurs, courriel unique, mot de passe haché, rôle
`projects`	Projets organisationnels, propriétaire, statut
`project_members`	Table d'association utilisateur ↔ projet, avec rôle
`meetings`	Réunions liées à un projet, cycle de statut, chemin audio
`participants`	Participants d'une réunion, hôte ou invité
`transcriptions`	Texte brut produit par Whisper, un par réunion
`decisions`	Décisions extraites par le LLM, contexte, confiance, statut
`mtg_memory_chunks`	Segments de texte vectorisés (768 dimensions) pour le RAG
`alerts`	Conflits entre décisions (structure prête, non encore alimentée)
`tasks`	Tâches assignées, liées à une réunion et/ou un projet

L'extension `vector` (pgvector) est activée sur la colonne `embedding` de `mtg_memory_chunks`, indexée par un index `HNSW` sur l'opérateur de distance cosinus, garantissant des recherches de similarité en temps sous-linéaire même lorsque le volume de segments augmentera.

7 Tests unitaires

7.1 Stratégie de test

La suite de tests repose sur **Jest 30**, exécutée avec le flag `-experimental-vm-modules` (nécessaire pour les modules ES natifs) via `dotenv-cli`, qui injecte les variables du fichier `.env.test` avant le lancement — ce fichier définit notamment un port, un secret JWT et des clés d'API distincts de l'environnement de développement.

À mi-parcours, la couverture se concentre sur les **fonctions utilitaires pures**, choisies en priorité car elles ne dépendent d'aucune ressource externe (base de données, réseau) et s'exécutent donc rapidement et de façon déterministe. La dépendance `supertest` est déjà installée en prévision des tests d'intégration des endpoints HTTP, prévus au bloc 17 — aucun test d'intégration n'existe encore à ce stade, ce qui est cohérent avec le statut *Todo* de ce bloc.

Table 4 – Suite de tests unitaires au mi-parcours

Fichier	Module testé	Cas
`hash.utils.test.js`	Hachage et vérification de mot de passe (`bcryptjs`)	5
`jwt.utils.test.js`	Génération et vérification de jetons JWT	6
`chunk.utils.test.js`	Découpage de texte en segments chevauchants (RAG)	6
Total	**3 suites, 17 cas de test**	**17**

7.2 Exemples de code

Le premier extrait teste `splitIntoChunks`, la fonction pure responsable du découpage des transcriptions en segments chevauchants avant vectorisation (cœur de l'étape d'indexation du pipeline RAG, figure 13).

Listing 2 – Extrait réel — tests/unit/chunk.utils.test.js

```
describe('chunk.utils', () => {
  describe('splitIntoChunks', () => {
    test('doit retourner un tableau vide pour un texte vide', () => {
      expect(splitIntoChunks('')).toEqual([]);
      expect(splitIntoChunks(null)).toEqual([]);
    });

    test('doit decouper un texte long avec le bon pas (taille - chevauchement)', () => {
      const chunks = splitIntoChunks(makeText(1200), 500, 100);
      expect(chunks.map((c) => c.charStart)).toEqual([0, 400, 800]);
    });

    test('la fin d\'un chunk doit se retrouver au debut du suivant', () => {
      const chunks = splitIntoChunks(makeText(1200), 500, 100);
      expect(chunks[0].text.slice(-100)).toBe(chunks[1].text.slice(0, 100));
    });

    test('doit lancer une erreur si le chevauchement est >= a la taille', () => {
      expect(() => splitIntoChunks(makeText(1000), 100, 100)).toThrow();
    });
  });
});
```

Le second extrait teste l'unicité du sel de hachage `bcrypt` — une propriété de sécurité

essentielle (deux hachages du même mot de passe ne doivent jamais être identiques), impossible à vérifier par simple lecture du code et donc particulièrement représentative de la valeur des tests automatisés.

Listing 3 – Extrait réel — tests/unit/hash.utils.test.js

```
describe('hashPassword', () => {
  test('doit retourner un hash bcrypt (commence par \$2b\$)', async () => {
    const hash = await hashPassword('test123');
    expect(hash).toMatch(/^\$2b\$/);
  });

  test('deux hashes du meme mot de passe doivent etre differents (salt)', async () => {
    const hash1 = await hashPassword('memeMotDePasse');
    const hash2 = await hashPassword('memeMotDePasse');
    expect(hash1).not.toBe(hash2);
  });
});

describe('comparePassword', () => {
  test('doit retourner false pour un mauvais mot de passe', async () => {
    const hash = await hashPassword('bonMotDePasse');
    const result = await comparePassword('mauvaisMotDePasse', hash);
    expect(result).toBe(false);
  });
});
```

Le troisième extrait teste la robustesse de `verifyToken` face à un jeton falsifié — une vérification de sécurité directement liée au diagramme de séquence d'authentification (figure 12).

Listing 4 – Extrait réel — tests/unit/jwt.utils.test.js

```
test('doit lever une erreur pour un token modifie', () => {
  const token = generateToken(payload);
  const tokenModifie = token.slice(0, -5) + 'xxxxx';
  expect(() => verifyToken(tokenModifie)).toThrow();
});
```

8 Blocs complétés — synthèse de la trace d'ingénierie

8.1 Bloc 01 — Infrastructure et configuration

Mise en place du dépôt, de `docker-compose.yml` (image `pgvector/pgvector:pg16` avec *healthcheck*), du squelette Express (`app.js`, `server.js`) et du schéma SQL initial. Les scripts `start.ps1` / `start.sh` automatisent la séquence complète de démarrage depuis un `git clone` propre.

8.2 Bloc 02 — Authentification JWT

Endpoints `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`. Hachage `bcryptjs`, jetons JWT sans état, middleware `authMiddleware` vérifiant à la fois la validité du jeton et l'état actif du compte (`is_active`) à chaque requête protégée.

8.3 Bloc 03 — Gestion des projets (CRUD)

Endpoints `/api/projects`. La création d'un projet (`createProjectWithOwner`) est transactionnelle : l'insertion du projet et l'ajout du créateur comme membre avec le rôle `owner` réussissent ou échouent ensemble. Seul le propriétaire peut modifier le projet ou son statut (vérification explicite `project.owner_id !== req.user.id`).

8.4 Bloc 04 — Gestion des réunions (CRUD)

Endpoints `/api/meetings`. La création (`createMeetingWithHost`) ajoute automatiquement le créateur comme participant `host`. Toutes les requêtes de lecture filtrent par appartenance de l'utilisateur au projet (`INNER JOIN project_members`), garantissant l'isolation des données entre équipes.

8.5 Bloc 05 — Transcription audio (Whisper)

Endpoint `POST /api/transcriptions/upload`, fusionné via la *pull request* #12. Le middleware `upload.middleware.js` (Multer) valide le type MIME audio et limite la taille du fichier (`MAX_FILE_SIZE_MB`). Le service `whisper.service.js` invoque le binaire `whisper` en sous-processus (`child_process.spawn`), configuré pour l'exécution CPU (`-device cpu -fp16 False`) afin de fonctionner sans GPU dédié.

8.6 Bloc 06 — Extraction des décisions (Gemini)

Endpoint `POST /api/decisions/extract/:meetingId`. Le prompt d'extraction (`ia.service.js`, fonction `extractDecisions`) impose un format de sortie JSON strict à cinq champs (`content`, `context`, `responsible`, `alternatives`, `confidence`), parsé après nettoyage des délimiteurs Markdown éventuels renvoyés par le modèle. Les décisions sont persistées dans une transaction qui fait également passer la réunion au statut `done`.

8.7 Bloc 07 — Mémoire vectorielle (RAG)

Trois opérations composent le pipeline RAG, implantées dans `memory.service.js` et `memory.model.js` : indexation (découpage, *embedding*, insertion transactionnelle avec désactivation logique des anciens segments lors d'une réindexation), recherche (requête `ORDER BY embedding <=> $1` avec filtrage par seuil de similarité), et consommation du contexte récupéré par le LLM pour la génération de réponse (section 5.4.3).

8.8 Bloc 10 — Frontend : authentification et navigation

Fusionné via la *pull request* #19. Structure complète du routage React Router (`App.jsx`) : routes publiques (`/login`, `/register`) et routes protégées regroupées sous `ProtectedRoute`, qui redirige vers `/login` tant qu'aucun utilisateur n'est résolu. Le jeton est persisté dans `localStorage` sous la clé `veridian_token`, lue par `api/client.js`.

8.9 Bloc 11 — Frontend : tableau de bord et pages de données

Fusionné via la *pull request* #24. Sept pages routées (`DashboardPage`, `MeetingsPage`, `ProjectsPage`, `TasksPage`, `MemoryPage`, `LoginPage`, `RegisterPage`) construites avec des données statiques le temps de stabiliser la structure visuelle avant de les relier à l'API réelle au bloc suivant.

Observation de code

Le fichier `HomePage.jsx` est importé dans `App.jsx` mais n'est rattaché à aucune route active — un résidu probable de l'échafaudage initial de Vite, candidat naturel à un nettoyage lors du bloc 13 ou 17.

8.10 Bloc 12 — Connexion du frontend à l'API réelle (en cours)

Développement terminé sur la branche dédiée `bloc-12` (commit `0e73ad1`, *« connecter toutes les pages frontend à l'API réelle »*), fusion vers `main` non encore effectuée au

moment de la rédaction. Les captures d'écran de la section 3 en attestent directement : les pages affichent des états vides authentiques plutôt que des gabarits statiques.

Ce bloc a également livré, en support direct des pages déjà construites au bloc 11, la ressource `tasks` qui n'existait pas encore côté backend : migration `003_tasks.sql`, `task.model.js`, `task.controller.js` et `task.routes.js`, avec l'endpoint `PATCH /api/tasks/:id/s` permettant de faire transiter une tâche entre `todo`, `in_progress` et `done` sans rechargement de page.

Le code actuel confirme également la résolution de trois incohérences qui, selon la pratique courante sur ce type d'intégration, se manifestent typiquement lors du branchement d'un frontend construit sur données statiques à une API réelle :

Cohérence vérifiée — clé de jeton

`services/api.js` et `context/AuthContext.jsx` lisent désormais le jeton via les mêmes fonctions centralisées (`getToken` / `clearToken` de `api/client.js`), sous la même clé `localStorage 'veridian_token'`.

Cohérence vérifiée — préfixe d'URL

`services/api.js` définit `BASE_URL = 'http://localhost:3000/api'`, incluant le préfixe `/api` sous lequel toutes les routes backend sont montées (`app.js`).

Cohérence vérifiée — CORS

`app.js` déclare un middleware CORS explicite autorisant `http://localhost:5173` et gérant les requêtes préliminaires `OPTIONS`, condition nécessaire à tout appel *cross-origin* du frontend Vite vers le backend Express.

9 Problèmes rencontrés

Cette section documente les difficultés techniques réelles rencontrées au cours des premiers blocs, ainsi que les solutions apportées, vérifiées contre l'état actuel du code. Elle alimentera la section correspondante du rapport final (TN4).

Table 5 – Journal des problèmes rencontrés

Bloc	Problème	Solution
07	La dimension des vecteurs d'*embedding* (1536, héritage d'une première intégration OpenAI) ne correspondait pas à celle produite par l'API Gemini (768), provoquant une erreur de type pgvector.	Migration corrective `001_update_vector_dimension.sql`, puis redéfinition complète de la table sous le nom `mtg_memory_chunks` dans la migration `007`.
07 / continu	L'API Gemini applique des limites de débit (429) et peut retourner des erreurs transitoires de surcharge (503) ou de modèle introuvable (404) lorsqu'un modèle est retiré du service.	Fonction `generateWithFallback` (`config/gemini.js`) : bascule automatique sur une liste ordonnée de modèles de secours, et `embedText` réessaie jusqu'à 3 fois avec un délai croissant pour les erreurs 429/503.
12	Avant la centralisation observée dans le code actuel, deux clés `localStorage` distinctes (`'token'` côté service API, `'veridian_token'` côté contexte d'authentification) auraient provoqué une déconnexion silencieuse à chaque appel.	Les deux modules importent désormais `getToken()` / `clearToken()` depuis `api/client.js`, source unique de vérité pour le jeton.
12	L'absence d'en-têtes `Access-Control-Allow-*` aurait bloqué silencieusement, côté navigateur, toute requête du frontend (`:5173`) vers le backend (`:3000`).	Middleware CORS manuel ajouté dans `app.js`, en amont de toutes les routes, avec court-circuit explicite des requêtes `OPTIONS`.
Démarrage	La redirection PowerShell `<` n'est pas supportée pour passer un fichier SQL à `docker exec`.	Utilisation de `cmd` ou de `Get-Content fichier.sql | docker exec -i ...` à la place.
Démarrage	Docker Desktop ne démarre pas automatiquement au démarrage de Windows malgré la configuration de `start.ps1`.	Démarrage manuel requis avant l'exécution du script ; cause précise non identifiée (délai de session Windows ou conflit avec la séquence de démarrage automatique).

10 Avancement et planification de la seconde moitié

10.1 Métriques d'avancement

Table 6 – Métriques du dépôt GitHub au mi-parcours

Indicateur	**Valeur**
Blocs fusionnés dans `main`	9 / 18 (01–07, 10, 11)
Bloc en fin de développement (branche dédiée)	1 (bloc 12)
Blocs partiellement livrés	1 (bloc 09 — chat opérationnel, export PDF à faire)
Pull requests fusionnées identifiées	#12, #19, #24
Branches distantes actives (non supprimées)	6
Tables PostgreSQL	10
Index explicites (dont 1 HNSW vectoriel)	11
Endpoints REST applicatifs	26 (+ 1 `/api/health`)
Pages frontend effectivement routées	7
Suites de tests unitaires	3 fichiers, 17 cas
Fichiers de migration SQL versionnés	3

10.2 Plan des blocs restants

Table 7 – Planification de la seconde moitié du projet

Bloc	Titre	Objectif principal
08	Contradictions & Alertes	Relier `detectContradiction()` à l'extraction de décisions ; persister dans `alerts` ; exposer un endpoint de consultation
09	Compléter Chat & Export PDF	Finaliser l'export PDF des décisions par projet (le chat RAG est déjà fonctionnel)
12	Finaliser API Connect	Fusionner la *pull request* du bloc 12 dans `main`
13	Frontend Formulaires	CRUD complet depuis l'interface : projets, réunions, tâches
14	Frontend Réunion Détaillée	Lecteur audio, transcription affichée, décisions liées
15	Décisions & Contradictions	Vue agrégée des décisions, alertes de contradiction visibles
16	Calendrier projet	Vue mensuelle des réunions par projet
17	Tests et qualité	Tests d'intégration (`supertest`) des endpoints critiques : auth, transcription, mémoire RAG
18	Déploiement et docs finales	Guide de déploiement, rapport TN4, version de démonstration

10.3 Risques résiduels

— **Performance de Whisper** : le modèle *small* traite l'audio en temps sensiblement supérieur au temps réel sur le matériel disponible (exécution CPU sans accélération). Pour des réunions longues, le délai de transcription peut devenir perceptible ; une file d'attente asynchrone est envisagée si le besoin se confirme.

— **Quota API Gemini** : le palier gratuit impose des limites de requêtes par minute. Le mécanisme de nouvelle tentative (`embedText`) et de bascule de modèle (`generateWithFallback`) atténue ce risque sans l'éliminer en cas de démonstration intensive.

— **Bloc 08 non débuté côté persistance** : la logique de détection de contradiction existe mais n'est reliée à aucun flux applicatif ; le risque est de sous-estimer l'effort d'intégration (transaction combinée extraction + détection + alerte) lors de la planification du bloc.

— **Docker Desktop au démarrage** : un démarrage manuel préalable reste nécessaire dans certains environnements Windows malgré `start.ps1`.

11 Conclusion

À mi-parcours, le projet Veridian dépasse ses objectifs intermédiaires sur le plan technique : neuf blocs sur dix-huit sont fusionnés dans `main`, un dixième est en fin de développement, et le pipeline central de l'application — enregistrement, transcription, extraction de décisions, indexation vectorielle, interrogation en langage naturel — fonctionne de bout en bout, comme en attestent le prototype capturé à la section 3 et les diagrammes de séquence qui en détaillent le fonctionnement interne.

La modélisation UML présentée dans ce rapport (cas d'utilisation, classes, activité, trois diagrammes de séquence, état, composants, déploiement) a été construite à partir du code réellement en place plutôt que d'une intention de conception, ce qui en garantit la fidélité pour la suite du projet. Les blocs restants (08, 09 à compléter, 12 à fusionner, puis 13 à 18) portent principalement sur la fermeture de la boucle de vérification des décisions, l'expérience utilisateur du frontend, la qualité logicielle par les tests d'intégration, et la documentation finale.

La trace d'ingénierie accumulée dans le dépôt GitHub — commits, *pull requests*, historique de fusion — démontre un développement discipliné et traçable, conforme aux pratiques attendues au niveau master, y compris dans la transparence apportée aux écarts entre la planification initiale et l'exécution réelle (renumérotation de blocs fusionnés ensemble, mise à jour de la contrainte `dotenv`, bloc en statut *Todo* sur le tableau malgré un développement avancé).

Références

Atlassian. (2024). Feature Branch Workflow [Consulté en 2024].

Auth0 by Okta. (2023). JSON Web Token Introduction [Consulté en 2024].

Fowler, M. (2018). *Refactoring : Improving the Design of Existing Code* (2e éd.). Addison-Wesley Professional.

Google DeepMind. (2024). Gemini 1.5 : Unlocking multimodal understanding across millions of tokens of context [Consulté en 2024].

Newman, S. (2021). *Building Microservices : Designing Fine-Grained Systems* (2e éd.). O'Reilly Media.

pgvector contributors. (2024). pgvector : Open-source vector similarity search for Postgres [Consulté en 2024].

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. *arXiv preprint arXiv :2212.04356*. https://arxiv.org/abs/2212.04356